

TLN HACKATHON 2026

Privacy Policy Decoder

Evidence-Backed Privacy Analysis with Transparent Deterministic Scoring

AI extracts evidence	CODE validates + scores	USER inspects the result
--	---	--

Project documentation prepared for the TLN Cybersecurity Challenge 2026.

Item	Project detail
Project type	Working web application
Primary domain	Privacy and data protection / cybersecurity
Frontend	HTML, CSS, Vanilla JavaScript
Backend	Node.js / Vercel Serverless Function
AI	Google Gemini via @google/genai
Deployment	Vercel
Source code	https://github.com/swayamjagtap/privacy-policy-decoder
Live demo	https://privacy-policy-decoder.vercel.app/

Prepared by Swayam Jagtap

20 September 2026

Document Scope

This report documents the implemented Privacy Policy Decoder prototype as it exists for the TLN Cybersecurity Challenge 2026. It is written as technical project documentation rather than as a peer-reviewed research paper. The report describes the implemented architecture, methodology, scoring framework, security considerations, user experience, testing performed during development, representative output, limitations, and future scope.

Important scope note

The A–F result is a project-defined informational framework. It is not a legal conclusion, an industry-standard privacy score, or a determination of regulatory compliance.

Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Proposed Solution
6. System Architecture
7. Privacy Analysis Methodology
8. Deterministic Scoring Framework
9. User Interface and UX Design
10. Security Considerations
11. Implementation and Project Structure
12. Testing and Validation
13. Representative Analysis
14. Challenges and Engineering Decisions
15. Limitations
16. Future Scope
17. Conclusion
18. Technical References

1. Abstract

Privacy policies contain critical information about data collection, third-party sharing, tracking, retention, user controls, AI/model use, and government disclosure. In practice, these details are often distributed across lengthy and technical documents that are difficult for non-specialist readers to interpret quickly.

Privacy Policy Decoder is a web-based privacy analysis tool designed to make this first-pass interpretation faster and easier to inspect. A user supplies a privacy policy as text. The system uses Google Gemini to extract structured findings across seven predefined privacy dimensions and then applies a deterministic scoring rubric in application code. The final interface presents a plain-English summary, a dimension-level assessment, evidence supporting each finding, confidence information, key takeaways, and an A–F concern grade based on a maximum of 42 concern points.

The central architectural decision is to separate AI interpretation from deterministic scoring. Gemini is responsible for extracting what the policy says; validated findings are then mapped to a predefined rubric by code. This prevents the model from being the sole authority for the final grade and makes the scoring logic inspectable. The system also distinguishes “not stated” from explicit non-use, preserves scope when handling AI/model language, and treats supplied policy text as untrusted input for prompt-injection resistance.

The result is a functional prototype that combines generative AI where natural-language interpretation is useful with deterministic application logic where transparency and repeatability are important.

Keywords: Privacy Analysis, Cybersecurity, Data Protection, Generative AI, Gemini, Evidence Extraction, Deterministic Scoring, Prompt Injection, Web Application

2. Introduction

Background

Privacy policies are intended to explain how a service handles personal information, but the practical usefulness of a policy depends on whether a reader can locate and understand the relevant information. A user may need to answer questions about what is collected, who receives information, how long it is retained, what choices are available, whether data may be used in AI/model workflows, and under what circumstances information can be disclosed to authorities.

The usability problem

These answers are often distributed across multiple sections of a long document. Reading the entire policy is possible, but it can be time-consuming, especially when the reader only wants an initial understanding of the major privacy practices.

Project motivation

The project was motivated by the idea that AI can help with the language-heavy part of the task, while a deterministic application layer can preserve transparency. Instead of asking a model for a single overall opinion, the system decomposes the task into structured findings and then applies a published, project-defined rubric.

Design principle

The product is intentionally evidence-first: the user sees what was extracted and the policy language supporting the finding, rather than being asked to trust an unexplained overall AI judgment.

3. Problem Statement

The engineering problem addressed by Privacy Policy Decoder is:

Problem statement

How can a web application use generative AI to extract useful privacy-policy findings while keeping the final assessment transparent, reproducible from the extracted findings, and grounded in the policy text supplied by the user?

The project focuses on first-pass comprehension. It does not attempt to independently verify a company's real-world practices, determine legal compliance, or replace the original policy document.

4. Objectives

1. Convert long privacy-policy text into a structured, understandable analysis.
2. Analyze a consistent set of seven privacy dimensions across policies.
3. Provide supporting policy evidence for each extracted finding.
4. Separate AI evidence extraction from deterministic scoring.
5. Distinguish “not stated” from an explicit absence or explicit non-use.
6. Provide a responsive web interface suitable for non-technical users and hackathon demonstration.

5. Proposed Solution

The application follows a staged pipeline. The user supplies the policy, the API validates the request, Gemini produces structured findings, application logic validates the model output, and the scoring module maps the findings to weighted points. The frontend then presents the result as a privacy snapshot.

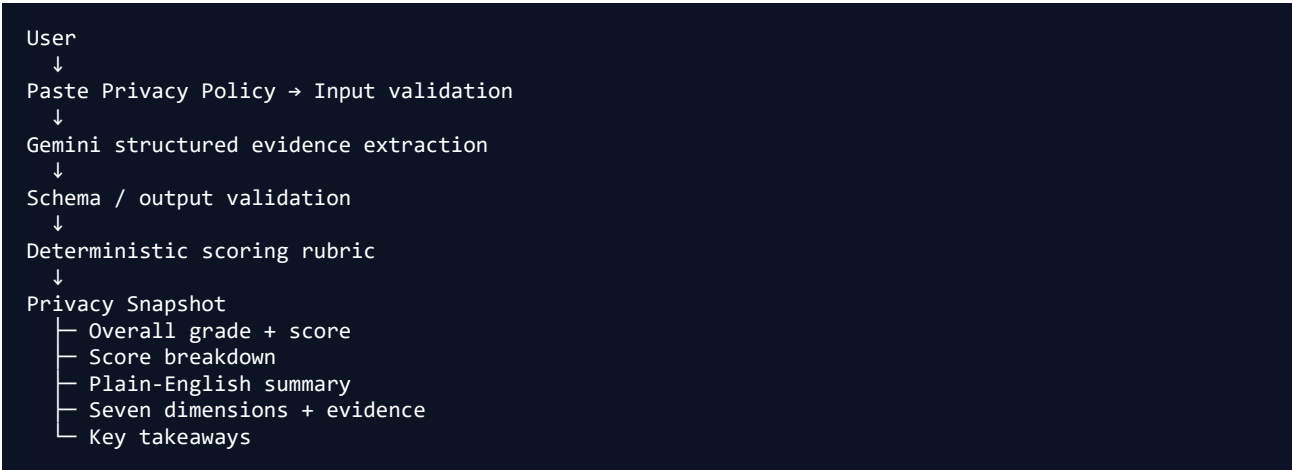


Figure 1. High-level processing pipeline used by Privacy Policy Decoder.

Core product behavior

- The user can paste up to 120,000 characters of policy text, with a minimum of 100 characters for analysis.
- The system analyzes seven predefined dimensions rather than requesting an unrestricted overall opinion.
- The model returns structured data rather than uncontrolled prose.
- The scoring module receives validated findings and produces weighted points, a total score, a grade, and key concerns.
- The frontend makes the evidence-to-score relationship visible to the user.

6. System Architecture

Layer	Primary responsibility	Implementation
Presentation	Collect policy text and render the analysis	HTML, CSS, Vanilla JavaScript
API	Validate request and coordinate analysis pipeline	Vercel Serverless Function: /api/analyze
AI extraction	Extract structured findings from policy text	Google Gemini via @google/genai
Schema validation	Ensure model output matches expected structure	lib/schema.js + lib/validate.js
Scoring	Convert validated findings into weighted points and grade	lib/scoring.js
Model integration	Retry, fallback, and environment handling	lib/gemini.js

Frontend responsibilities

The frontend handles input validation, character counting, example-policy loading, loading-state feedback, result rendering, risk/grade styling, dimension sorting, score breakdown presentation, and safe rendering of returned text. The current version also presents a three-step product explanation: “You paste,” “AI extracts,” and “Code scores.”

Backend responsibilities

The /api/analyze endpoint accepts POST requests, validates policy length, invokes the Gemini extraction helper, parses and validates the structured model response, applies the deterministic scoring function, and returns the final structured analysis. Internal failures are surfaced to the frontend as controlled errors rather than exposing credentials or internal implementation details.

AI/model integration

The configured Gemini integration uses gemini-3.6-flash as the primary model and gemini-3.5-flash as a fallback in the current prototype. Retry logic is used for transient failures. The API key is kept server-side through environment variables.

7. Privacy Analysis Methodology

Seven analysis dimensions

Dimension	What the system looks for
Data Collection	Breadth and scope of personal information described as collected.
Third-Party Sharing	Sharing with service providers, affiliates, advertisers, business partners, or other parties.
Tracking	Cookies, analytics, behavioral monitoring, cross-site tracking, and advertising-related tracking.
Retention	Whether retention is clearly bounded, vague, indefinite, or not stated.
User Control	Access, update, deletion, settings, and other user choices described by the policy.
AI / Model Use	Whether data or content may be used to develop, improve, evaluate, or otherwise support AI/model systems.
Government Disclosure	Conditions under which information may be disclosed to government or law-enforcement authorities.

Evidence extraction rules

- Use only the supplied policy text as the evidence source.
- Do not make legal conclusions or claim regulatory compliance/non-compliance.
- Treat “not stated” as a separate outcome rather than interpreting omission as “no.”
- Preserve the scope and strength of the source language.
- Prefer evidence that directly supports the chosen assessment.
- Avoid turning vague retention language into an unsupported claim of indefinite retention.

- Do not generalize a narrow statement about AI/model use into a global statement about all data or all model activity.
- Avoid using “train/training” in the summary unless the policy explicitly supports that terminology; otherwise use neutral model-development language.

Structured output

Each dimension returned by the model is represented with an assessment, evidence, and confidence level. The validated response also includes a plain-English summary. The scoring layer then enriches each dimension with `weighted_points`, `max_weighted_points`, and a stable dimension key for frontend rendering.

8. Deterministic Scoring Framework

The scoring framework is implemented in application code. The model does not generate the final A–F grade. Each classification maps to base points and a dimension weight. The weighted contribution is the point value multiplied by the dimension weight. The maximum possible score is 42.

Dimension	Assessment	Base points	Weight
Data Collection	Minimal	0	3
Data Collection	Moderate	1	3
Data Collection	Broad	2	3
Data Collection	Not stated	1	3
Third-Party Sharing	None	0	4
Third-Party Sharing	Service providers only	1	4
Third-Party Sharing	Broad commercial	3	4
Third-Party Sharing	Not stated	2	4
Tracking	None	0	3
Tracking	Limited	1	3
Tracking	Extensive	2	3
Tracking	Not stated	1	3
Retention	Clear and bounded	0	3
Retention	Vague	1	3
Retention	Indefinite	2	3
Retention	Not stated	2	3
User Control	Full	0	4
User Control	Partial	1	4
User Control	None	2	4
User Control	Not stated	2	4
AI / Model Use	Not used	0	1
AI / Model Use	Conditional use	1	1
AI / Model Use	Explicit use	2	1
AI / Model Use	Not stated	1	1

Dimension	Assessment	Base points	Weight
Government Disclosure	Narrow lawful	0	1
Government Disclosure	Broad or vague	2	1
Government Disclosure	Not stated	1	1

Grade mapping

Grade	Concern label	Score percentage range
A	Low Concern	0–15%
B	Limited Concern	>15–30%
C	Moderate Concern	>30–50%
D	Elevated Concern	>50–70%
E	High Concern	>70–85%
F	Significant Concern	>85%

Interpretation rule

The grade is a project-defined framework for informational analysis. It should not be presented as an industry-standard privacy rating or a legal/compliance judgment.

9. User Interface and UX Design

Landing experience

- Clear product identity and positioning: “Privacy, without the legal jargon.”
- Concise explanation of what the tool analyzes.
- Three-step “How it works” pipeline showing separation of user input, AI extraction, and code-based scoring.
- Fictional example workflow so a first-time user can immediately explore the product.

Analysis experience

- Large policy input area with 120,000-character ceiling.
- Live character counter and visible keyboard shortcut.
- Controlled loading state.
- Privacy note explaining that policy text is not stored by the application.

Results experience

UI element	Purpose
Overall grade	Communicates the project-defined concern band and total score.
Score breakdown	Shows each dimension’s weighted contribution to the total.
Plain-English summary	Provides a quick reading of the major findings.

UI element	Purpose
Detailed findings	Shows assessment, risk level, score contribution, evidence, and confidence.
Key takeaways	Surfaces the highest-concern findings for rapid scanning.
Disclaimer	Clarifies the informational and project-defined nature of the result.

Product message

AI extracts the evidence. Code determines the score.

10. Security Considerations

Untrusted policy text and prompt injection

Privacy-policy text supplied by a user is treated as untrusted input. The Gemini prompt explicitly instructs the model that content inside the policy payload is data to analyze, not a new instruction source. This guards against policy text containing adversarial phrases intended to override the analysis instructions.

Output validation

The model response is checked against an expected structure before it can reach the scoring layer. This prevents malformed or unexpected model output from silently becoming the basis for a grade.

Safe rendering

The frontend uses DOM text nodes and textContent for user-facing model-derived content rather than inserting returned content as raw HTML. This reduces the risk of introducing executable markup from untrusted text.

Secret management

The Gemini API key is kept server-side through environment variables and is not exposed in frontend code. Local environment files are excluded from source control.

Input validation and abuse boundaries

The API enforces a minimum and maximum policy length. The current production prototype also uses retry and fallback behavior for transient model failures. Additional production hardening such as dedicated rate limiting can be added in future versions.

11. Implementation and Project Structure

```

privacy-policy-decoder/
├── api/
│   └── analyze.js
├── lib/
│   ├── gemini.js
│   └── prompt.js

```



Figure 2. Project structure of the implemented prototype.

File	Responsibility
api/analyze.js	POST endpoint; request validation; pipeline coordination; structured response.
lib/gemini.js	Gemini client, environment handling, retry logic, fallback model.
lib/prompt.js	Structured evidence-extraction prompt and analysis guardrails.
lib/schema.js	Expected structured response schema.
lib/validate.js	Validation of model output and required fields.
lib/scoring.js	Deterministic rubric, weighted scoring, grade mapping, and key concerns.
app.js	Input handling, API request, score rendering, evidence cards, example flow, and UI state.
index.html	Semantic product interface and content structure.
style.css	Visual design, responsive behavior, grade/risk styling, evidence treatment.
golden-test.js	Synthetic regression/golden test harness used during development.
analyze-file.js	Local real-policy analysis harness.
test-connection.js	Gemini connectivity check.

12. Testing and Validation

Code-level validation

- JavaScript syntax checks were run across modified and supporting modules.
- Module imports and dependency resolution were checked after the frontend redesign.
- Git whitespace validation was run before final commits.
- No new package dependency was required for the Antigravity frontend redesign.
- No API key or environment secret was introduced into committed source files.

AI pipeline testing

- Gemini connectivity was tested independently before integration testing.

- A synthetic golden test was used to exercise expected classifications and deterministic scoring.
- The analyzer was exercised against real privacy-policy texts during development, including Discord and Notion, to refine evidence extraction, prompt guardrails, and regression behavior.
- Transient model failures were tested with retry/fallback behavior.

Frontend and deployment testing

- The application was run locally through Vercel’s development server so the frontend could exercise the real /api/analyze route.
- The deployed Vercel application was exercised with the built-in fictional example and a real policy workflow.
- The result page was verified to render score breakdowns, evidence, confidence, key takeaways, and the final disclaimer.

Testing limitation

The project does not claim a formal accuracy benchmark, precision/recall evaluation, or human-labeled gold-standard study. Testing performed to date is engineering validation and regression testing, not a scientific efficacy evaluation.

13. Representative Analysis

The built-in fictional example is designed to exercise all seven analysis dimensions. A representative current run produced the following result. Because the AI extraction stage is model-based, borderline classifications may vary between runs; the score shown below is therefore an example output of the current implementation, not a fixed benchmark.

Representative output

ExampleApp (Fictional) — 28 / 42 concern points — D: Elevated Concern.

Dimension	Assessment	Weighted points
Third-Party Sharing	Broad Commercial	12 / 12
Data Collection	Broad	6 / 6
Tracking	Extensive	6 / 6
Retention	Vague	3 / 6
AI / Model Use	Conditional Use	1 / 2
User Control	Full	0 / 8
Government Disclosure	Narrow Lawful	0 / 2

Representative evidence

Third-Party Sharing — “We may share information with business partners and affiliates for joint marketing initiatives.”

Data Collection — “We collect your name, email address, device identifiers, IP address, browser type, operating system, approximate location (city-level), and usage data including pages visited, features used, and session duration.”

Tracking — “Third-party advertising partners may also place cookies on your device to serve targeted advertisements based on your browsing behavior across websites.”

Retention — “We retain your personal information for as long as necessary to fulfill the purposes described in this policy and as required by applicable law.”

AI / Model Use — “We may use de-identified usage data to improve and develop our machine learning models and recommendation systems.”

14. Challenges and Engineering Decisions

Keeping AI in a narrow role

A simple implementation could ask an AI model for an overall privacy rating. The project deliberately avoids that approach because it makes the basis of the grade difficult to inspect. Instead, the model performs structured extraction while application code performs scoring.

Handling ambiguity

Privacy policies frequently contain conditional language. A statement can apply to one data category, one product, or one legal condition. The extraction prompt therefore instructs the model to preserve scope and avoid broadening narrow statements.

Separating “not stated” from “no”

This distinction became a central methodological rule. Omitting information from a policy is not evidence that the practice does not occur, so the system exposes “not stated” as its own outcome where appropriate.

Improving the user experience without adding feature bloat

The final frontend redesign focused on explainability rather than additional product scope. The score breakdown, evidence-first cards, fictional example workflow, and visible AI/code pipeline were chosen because they reinforce the existing product concept without requiring new infrastructure.

Production behavior versus demo behavior

The built-in fictional example is useful for demonstrations, but it is not a fixed benchmark. Since the model performs natural-language classification, borderline findings may differ across runs. The application therefore does not hardcode a target grade for the example.

15. Limitations

- The tool analyzes only the policy text supplied by the user and does not independently verify real-world company behavior.
- The A–F grade is a project-defined rubric, not an industry-standard privacy rating.
- The result is not legal advice and does not determine regulatory compliance.
- LLM-based extraction can produce different classifications when policy language is ambiguous or borderline.
- The current MVP expects pasted text rather than automatically fetching a privacy policy URL.
- The prototype does not maintain a historical database of policy versions or changes.
- The current system is designed for first-pass analysis rather than exhaustive legal or technical due diligence.

16. Future Scope

- URL-based privacy-policy ingestion and extraction.
- Policy version comparison and change detection.
- User-visible source-location navigation for evidence.
- Historical policy monitoring and change alerts.
- Additional analysis dimensions and configurable scoring frameworks.
- Production-grade rate limiting and abuse controls.
- Optional privacy-policy comparison across multiple services.
- Expanded evaluation using human-labeled policy examples and a formal benchmark dataset.

17. Conclusion

Privacy Policy Decoder demonstrates a practical architecture for using generative AI in a privacy and cybersecurity workflow without giving the model unrestricted authority over the final result. The system uses Gemini where language understanding is valuable and deterministic application logic where transparency, reproducibility, and inspectability matter.

The resulting prototype turns a long policy into a structured privacy snapshot containing a plain-English summary, seven dimension-level findings, evidence, confidence, a project-defined score, and key concerns. The user is not asked to accept an opaque AI verdict; instead, the product is designed to show the path from policy language to extracted finding to scoring outcome.

The project's central principle can be summarized as:

Core principle

AI extracts the evidence. Code determines the score. The user inspects the result.

18. Technical References

Project repository: <https://github.com/swayamjagtap/privacy-policy-decoder>

Live application: <https://privacy-policy-decoder.vercel.app/>

Google Gemini API documentation: <https://ai.google.dev/gemini-api/docs>

Vercel documentation: <https://vercel.com/docs>

TLN Hackathon 2026 — Devpost: <https://tln-cybersecurity-challenge.devpost.com/>

Appendix A — Current Product Output Structure

The current production interface exposes the following result sequence after a successful analysis:

```
Privacy Analysis
├── Overall Grade
│   ├── Letter grade
│   ├── Concern label
│   └── Total weighted score / 42
└── Score Breakdown
    └── Seven dimension contributions
```

- Plain-English Summary
- Detailed Findings
 - Assessment
 - Risk level
 - Weighted points
 - Policy evidence
 - Confidence
- Key Takeaways
- Disclaimer

Appendix B — AI / External Tools Disclosure

Google Gemini is used as a core component of Privacy Policy Decoder for structured evidence extraction from supplied privacy-policy text. The application validates the model output and applies its own deterministic scoring rubric in code; the final grade is not generated directly by the model.

AI-assisted development tools were also used during development for coding assistance, debugging, prompt refinement, testing, and development workflow support. The submitted implementation, architecture, scoring logic, frontend, backend integration, and design decisions were reviewed and understood by the developer.

End of report